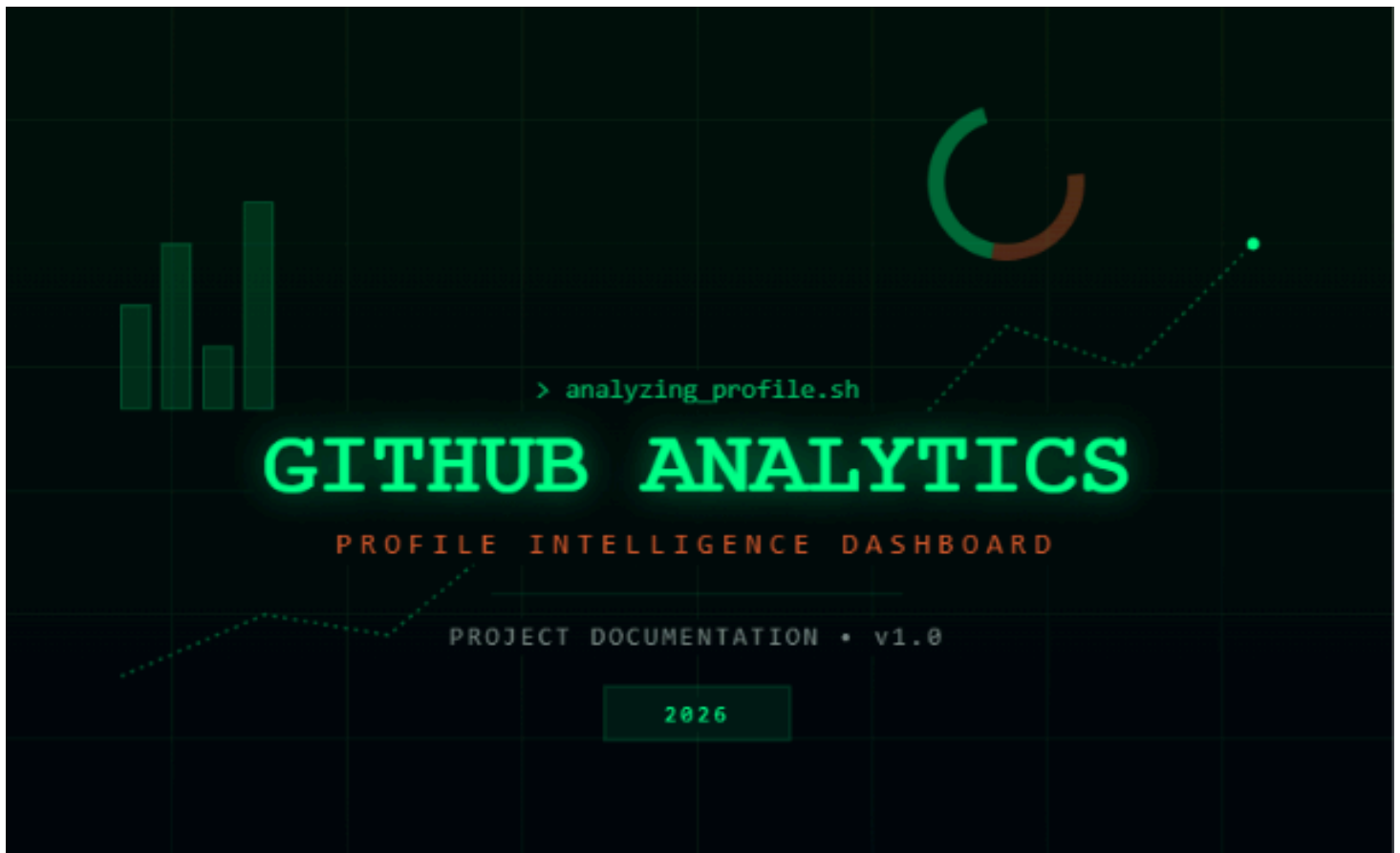


GitHub Analytics Dashboard

Project Planning & Requirements Document

Project Documentation • v1.0 • 2026



Group: ITC_SHR4_SWD2_S1

Track: React Frontend Web Developer

Team : 1.Norhan Elyan

2. Shehab Abdelrahman

3. Aya Gamal

4. Hassan Elshair

5. Alya Abuelmaati

01 Project Planning & Management

1.1 Project Proposal

GitHub Analytics Dashboard is a web application that allows users to visually and professionally analyze GitHub profiles. Users can enter any username to view full profile data including repositories, stars, and programming languages — with the ability to compare two users side by side.

1.2 Project Goal & Problem Statement

Main Goal	Provide a fast, visual analysis tool for GitHub profiles without requiring manual API calls or code
Problem Solved	The difficulty of comparing and analyzing GitHub profiles in a visual, easy-to-understand format
Value Added	An interactive dashboard showing detailed statistics and visual comparisons between developers
Target Audience	Developers, HR/Recruiters, tech students, hackathon organizers

Gantt Chart

Timeline overview based on Student deadlines (Feb– Jul 2026)

Phase / Task	Jan	Feb	Mar	Apr	May	Jun
Project Planning & Management						
Literature Review						
Requirements Gathering						
System Analysis & Design						
Implementation						
Final Presentation & Reports						

1.3 Task Assignment & Roles

Team Member	Role	Responsibilities
Norhan Elyan	Project Lead / FrontEnd	Project management, GitHub API integration, Compare feature, Header component
Alya Abuelmaati	Frontend Developer	App shell, tab navigation, layout, SearchBar component
Aya Gamal	Frontend Developer	Shared UI components, Terminal log component
Shehab Abdelrahman	Frontend Developer	UserCard component, Overview tab (profile + stars chart)
Hassan Elshair	Frontend Developer	Repos tab (repository table), Languages tab (pie chart & legend)

1.4 Risk Assessment & Mitigation Plan

Risk	Likelihood	Impact	Mitigation
GitHub API Rate Limit	High	High	Show a clear message and suggest adding a Personal Access Token
CORS when opened via file://	High	High	Run the project on a local server (Vite / npm)
User not found or private profile	Medium	Medium	Check response status and display a descriptive error message
Slow data load (100 repos)	Medium	Low	Add a visible loading state in the Terminal log
Old browser compatibility	Low	Low	Use Vite with automatic polyfills

--	--	--	--

Task Details

ID	Task	Assignee	Deadline
KAN-1	App shell, tab navigation & layout	Alya Abuelmaati	Mar 15, 2026
KAN-2	GitHub API integration & data fetching	Norhan Elyan	Mar 20, 2026
KAN-3	SearchBar component	Alya Abuelmaati	Mar 25, 2026
KAN-4	Shared UI components	Aya Gamal	Apr 1, 2026
KAN-5	Terminal log component	Aya Gamal	Apr 5, 2026
KAN-6	UserCard component	Shehab Abdelrahman	Apr 10, 2026
KAN-7	Overview tab — profile + stars chart	Shehab Abdelrahman	Apr 15, 2026
KAN-8	Repos tab — repository table	Hassan Elshair	Apr 20, 2026
KAN-9	Languages tab — pie chart & legend	Hassan Elshair	Apr 25, 2026
KAN-10	Header component	Norhan Elyan	Apr 30, 2026
KAN-11	Compare feature — Radar Chart & Head-to-Head	Norhan Elyan	May 10, 2026

--	--	--	--

1.5 Key Performance Indicators (KPIs)

Metrics used to measure the success of the GitHub Analytics Dashboard project

KPI	Target Metric	Measurement Method	Category
API Response Time	< 3 seconds	Browser DevTools / Network tab	Performance
GitHub API Rate Limit	< 60 req/hr	API response headers	Performance
Compare Mode Accuracy	100% correct score	Manual test with known profiles	Quality
Error Handling Coverage	All cases handled	Testing invalid/private usernames	Reliability
UI Responsiveness	Works on Desktop	Manual cross-browser testing	Usability
Code Modularity	Separate components/hooks/utils	Code review	Maintainability
Task Completion Rate	100% by deadline	Jira board status	Management
GitHub Commits	Regular commits	GitHub commit history	Collaboration

02 Requirements Gathering

2.1 Users / Stakeholders

Software Developer	Wants to analyze their profile, track statistics, and identify strengths
HR / Recruiter	Wants to quickly compare candidate profiles for a job position
Tech Student	Wants to see their level compared to other developers for learning purposes
Hackathon Organizer	Wants to assess participants' skill level through their GitHub profiles

2.2 User Stories

1. As a developer, I want to enter a username and view all my GitHub statistics so I can track my progress.
2. As a developer, I want to see the most used programming languages in my projects so I can identify my actual specialization.
3. As an HR professional, I want to compare two profiles simultaneously so I can choose the best candidate for a position.
4. As a user, I want to see a list of repositories with Stars / Forks / Language details so I can evaluate project quality.
5. As a user, I want to see a Radar Chart comparing two users across multiple dimensions so I can clearly understand the differences.
6. As a user, I want to see a Terminal log showing the status of data fetching so I know what is happening in the background.

2.3 Use Cases

UC#	Use Case	Main Steps	Expected Result
UC-01	Analyze Profile	Enter username → click ANALYZE → data loads	Profile Card + Charts displayed
UC-02	Compare Two Users	Enable Compare Mode → enter two usernames → analyze	Radar Chart + Winner card displayed
UC-03	Browse Repos	Click the Repos tab	Full table of all projects with details
UC-04	Analyze Languages	Click the Languages tab	Pie Chart + Language legend displayed
UC-05	Monitor Status	Watch Terminal log during data loading	Live log of each API request status

2.4 Functional Requirements

FR-01	The system must fetch user data from the GitHub API when a username is entered
FR-02	The system must display a Profile Card including: Avatar, Name, Bio, Repos, Stars, Forks, Followers, Following
FR-03	The system must display a Bar Chart of the top 8 repositories by star count
FR-04	The system must display a table of all repositories with Stars, Forks, Language, and Fork indicator
FR-05	The system must display a Pie Chart of programming language distribution with a legend
FR-06	The system must support Compare Mode to compare two users via Radar Chart and Head-to-Head bars
FR-07	The system must calculate a Winner Score based on: Stars×3 + Followers×2 + Repos + Langs×5
FR-08	The system must display a Terminal Log tracking the status of each API request (fetching, success, error)
FR-09	The system must display clear error messages when a user is not found or the connection fails

2.5 Non-Functional Requirements

Category	Metric	Details
Performance	Speed	Display results in under 3 seconds on a good connection
Performance	Data Fetching	Fetch first 100 repos in a single request to minimize API calls
Security	API Safety	No user data is stored — everything lives in component state only
Security	Rate Limiting	Show a clear message when the 60 req/hr limit is reached without a token
Usability	UI/UX	Simple interface with no instructions needed — clear placeholders in every input
Usability	Responsive	Fully functional on Desktop screens
Reliability	Error Handling	Every error case is caught and displayed in the Terminal log
Maintainability	Code Structure	Code organized into separate components, hooks, and utils folders

03 Features

3.1 Current Features

Feature	Description	Priority	Status
Search & Fetch	Fetch any GitHub user's data by username via the GitHub REST API	High	✓ Done
Profile Card	Display Avatar, Name, Bio, and all core stats	High	✓ Done
Top Repos Chart	Bar Chart of top 8 repos by star count	High	✓ Done
Repos Table	Detailed table of all projects with Stars / Forks / Language	High	✓ Done
Languages Pie Chart	Pie Chart of programming language distribution + legend	High	✓ Done
Compare Mode	Compare two users with an easy toggle switch	High	✓ Done
Head-to-Head Bars	Visual comparison of each metric between the two users	Medium	✓ Done
Winner Card	Calculate and display the winner based on a composite score	Medium	✓ Done
Terminal Log	Live log of every API request status in real time	Medium	✓ Done
Account Age	Shows account age in years and a Veteran badge for older accounts	Medium	✓ Done
Recent Searches	Quick-access chips for the most recently analyzed usernames	Medium	✓ Done
Repo Filter	Live text filter to search within a user's repository list	Medium	✓ Done

3.2 Upcoming Features (Planned)

Feature	Description	Priority	Status
Contribution Graph	Display daily contribution heatmap (GitHub activity graph)	High	 Planned
Pinned Repos	Show repositories pinned to the user's profile	High	 Planned
Language by Bytes	Analyze languages by actual byte count rather than repo count	Medium	 Planned
Export Profile	Export profile data as a PDF or screenshot	Low	 Planned
GitHub Token Support	Add Personal Access Token support to raise the API rate limit	Low	 Planned

04 System Analysis & Design

4.1 Software Architecture

The GitHub Analytics Dashboard follows a pure Frontend Architecture (Client-Only). There is no backend server or database — all data is fetched directly from the GitHub REST API v3 and rendered in the browser using React components.

Technology Stack

Layer	Technology	Purpose
UI Framework	React (Vite)	Component-based frontend rendering
Charting Library	Chart.js	Bar, Pie, and Radar charts
Styling	CSS Modules / Tailwind	Responsive, modular styling
Data Layer	GitHub REST API v3	Fetch user and repo data
State Management	React useState / useEffect	Local component state
Build Tool	Vite	Fast development server & bundling
Version Control	GitHub	Source code & CI hosting

Architecture Style

Component-Based Architecture: The UI is split into reusable React components (SearchBar, UserCard, Charts, TerminalLog), each responsible for a single concern. Data flows top-down via props from the App root component

4.2 Use Case Diagram

The diagram below identifies the system actors and their interactions with the GitHub Analytics Dashboard:

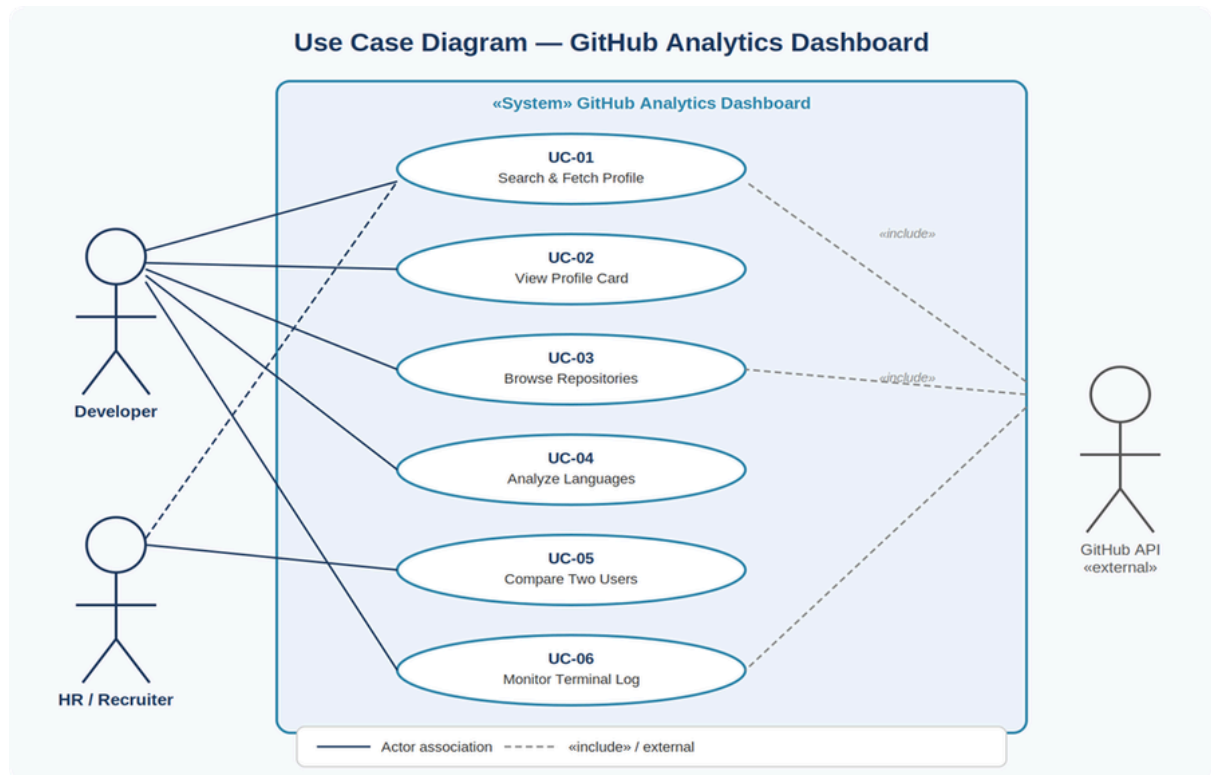


Figure 1: Use Case Diagram — Actors: Developer, HR/Recruiter, GitHub API (external)

Use Case Descriptions

- UC-01 — Search & Fetch Profile: User enters a GitHub username; system calls GitHub API and loads data.
- UC-02 — View Profile Card: System displays avatar, bio, repos, stars, forks, followers, following.
- UC-03 — Browse Repositories: User views a sortable table of all repositories with key metrics.
- UC-04 — Analyze Languages: System renders a Pie Chart of programming language distribution.
- UC-05 — Compare Two Users: User enables Compare Mode; system loads two profiles and renders Radar Chart + Winner Card.
- UC-06 — Monitor Terminal Log: System logs each API call status in real time in a terminal-style panel.

4.3 Sequence Diagram

The sequence diagram below shows the step-by-step interaction between system components during a typical profile analysis (UC-01):

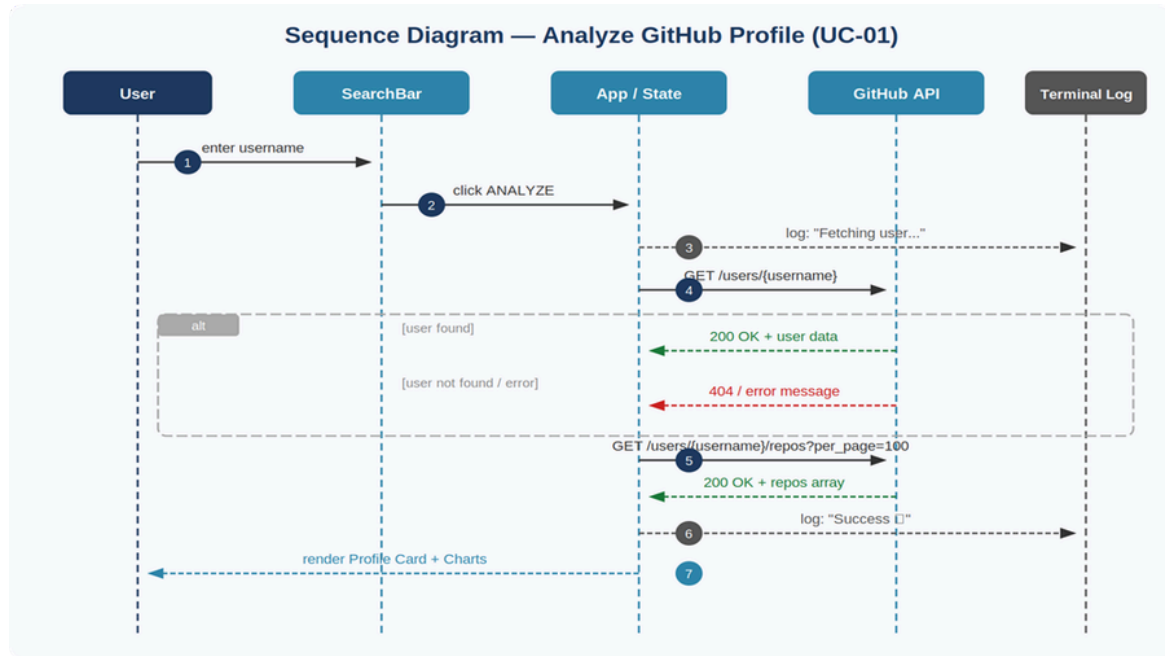


Figure 2: Sequence Diagram — Analyze GitHub Profile (UC-01)

4.4 Class Diagram

The class diagram defines the structure of the system, showing the main components, their attributes, methods, and relationships:

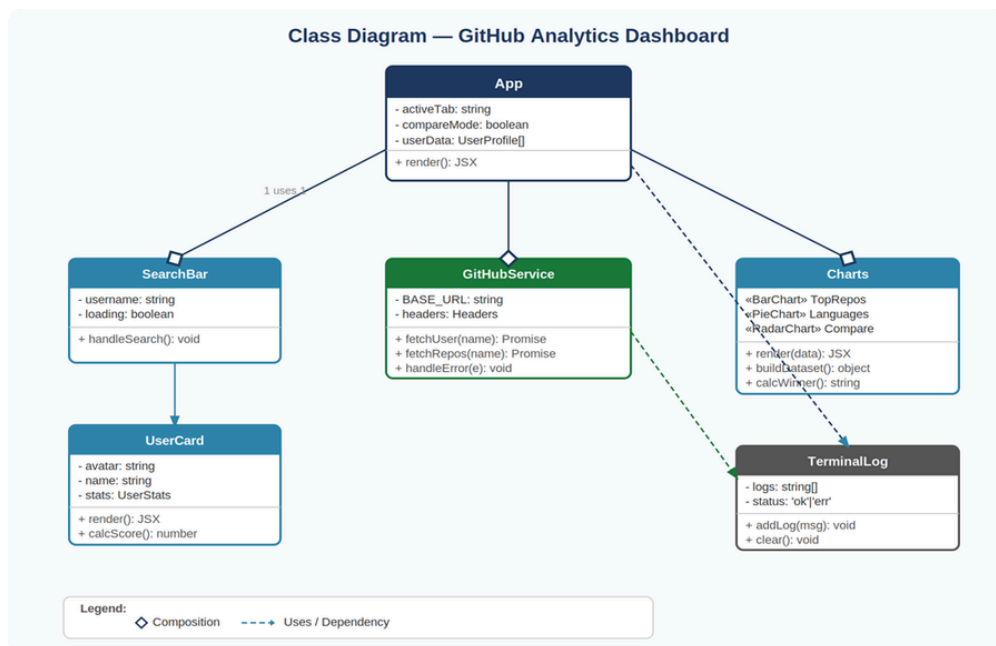


Figure 3: Class Diagram — System Structure

4.5 Activity Diagram

The activity diagram illustrates the complete user workflow from entering a username to viewing results, including the Compare Mode branch:

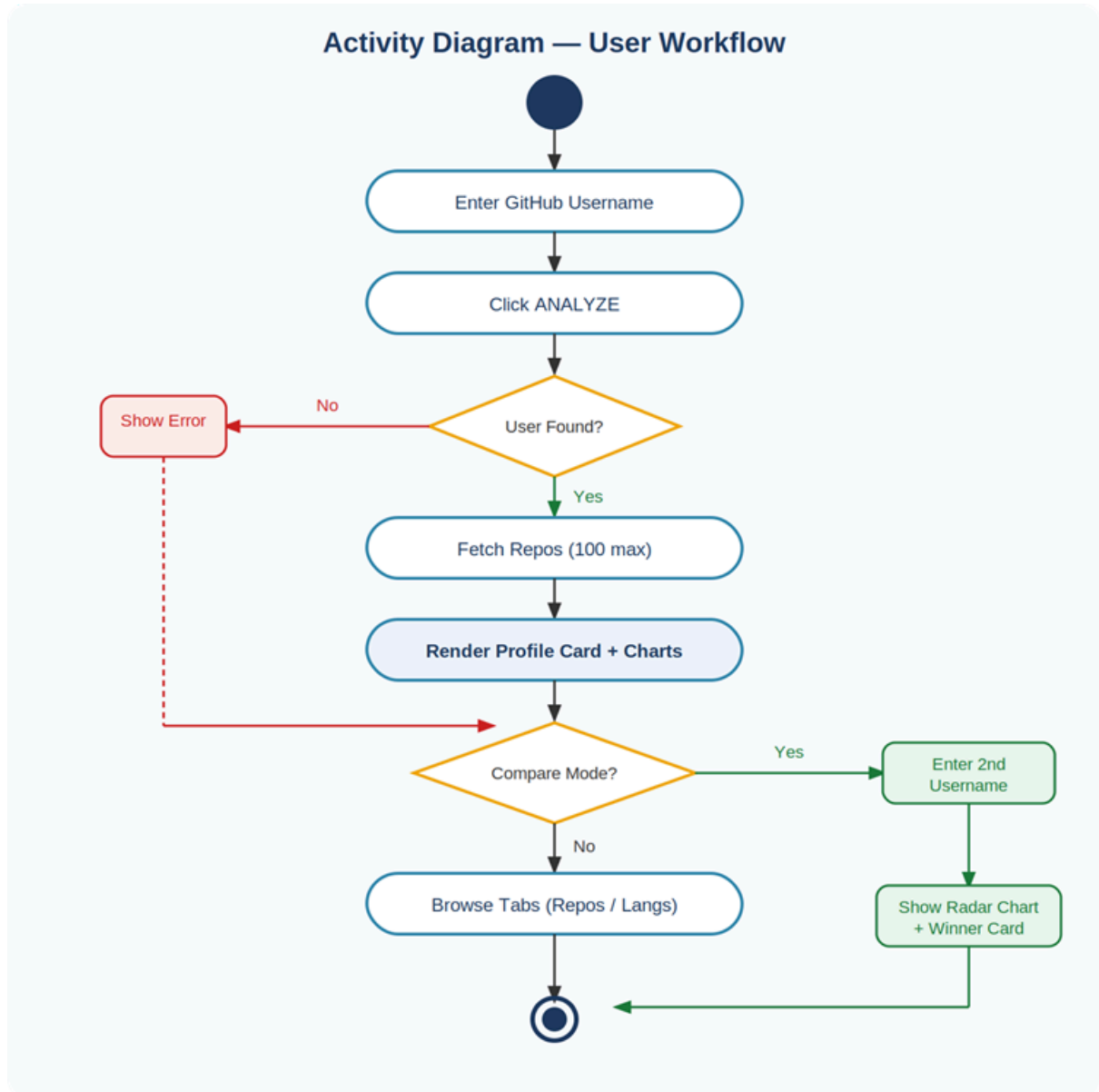


Figure 4: Activity Diagram — User Workflow

7. Component Diagram

The component diagram shows the high-level system components and their dependencies, including the external GitHub REST API:

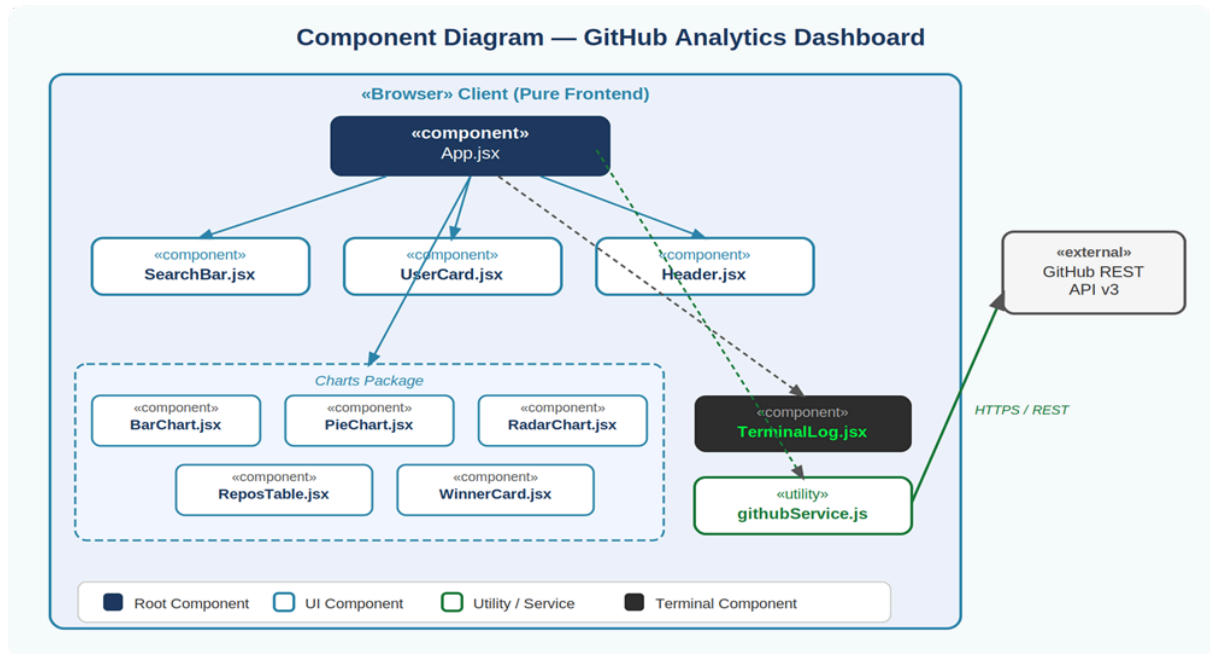


Figure 5: Component Diagram — System Components

05 Functional Specification & Test Cases

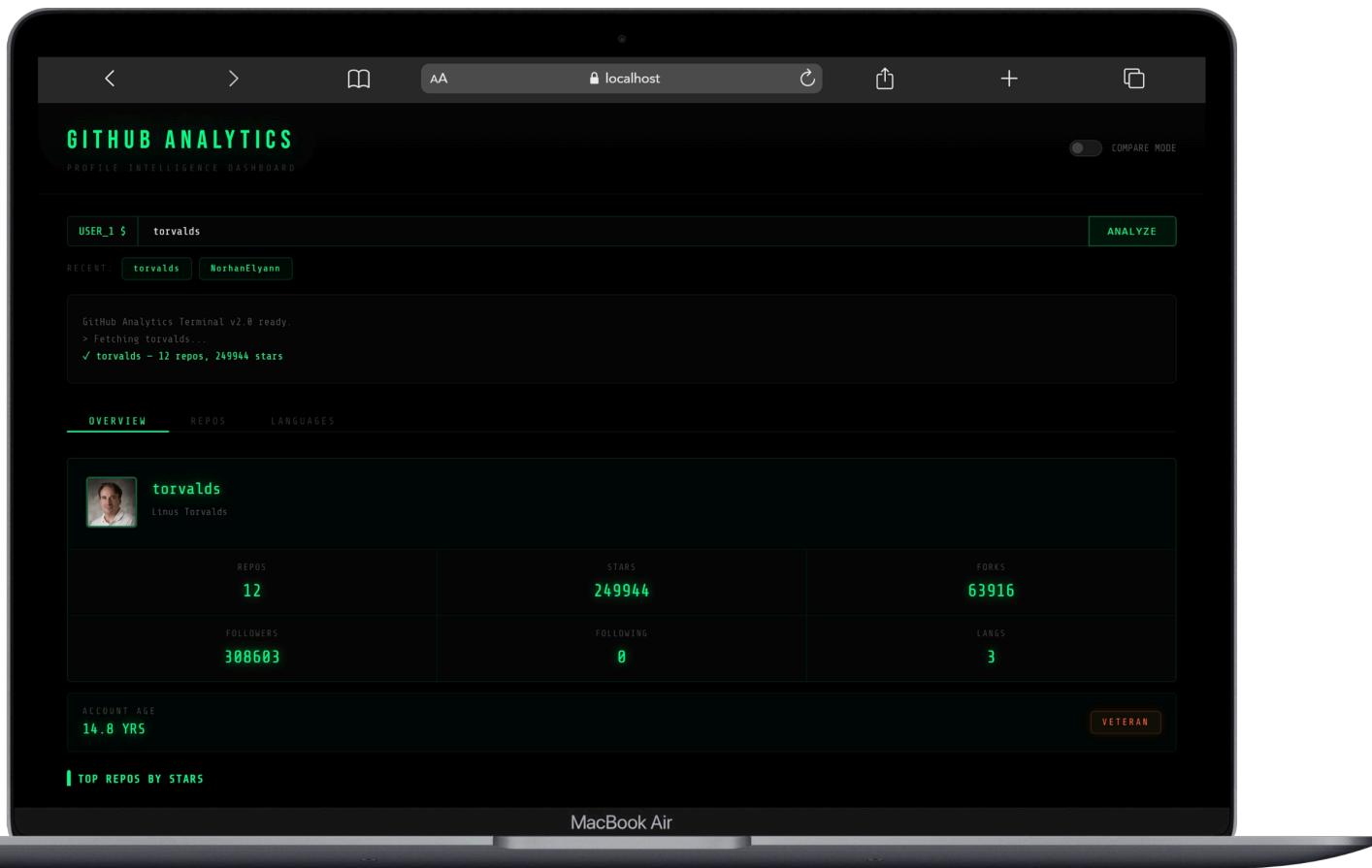
TC#	Req. Ref	Test Case Description	Execution Steps	Expected Result
TC-01	FR-01	Verify user data retrieval via GitHub API.	1. Enter a valid username. 2. Click Search or hit Enter.	System initiates fetching state and loads user profile data successfully.
TC-02	FR-02	Verify complete Profile Card rendering.	1. Inspect dashboard layout after a successful search.	Displays Avatar, Name, Bio, Repos, Stars, Forks, Followers, and Following.
TC-03	FR-03	Verify Bar Chart filters Top 8 Repositories.	1. Navigate to the charts/analytics section.	Renders sorted bar chart limited to exactly the top 8 starred repositories.
TC-04	FR-04	Verify Repositories Table & Fork tracking.	1. Open Repositories tab. 2. Check list for forked projects.	Renders dynamic table grid. Forked repositories display a custom hyphen (-) indicator.
TC-05	FR-05	Verify Donut Pie Chart & Legend alignment.	1. Open Languages tab. 2. Compare chart segments to side legend.	Donut chart renders smoothly. Legend items have colors matching the chart slices.

TC-06	FR-06	Verify layout shift in Compare Mode.	1. Input two usernames. 2. Trigger Compare mode.	Dashboard shifts to split-screen grid to compare both users side-by-side.
TC-07	FR-07	Verify algorithmic calculation of Winner Score.	1. Trigger compare mode with two users.	Evaluates custom score formula accurately and displays the winner badge.
TC-08	FR-08	Verify realtime state logging in Terminal Log.	1. Trigger a new search query. 2. Watch the bottom terminal console	Logs update dynamically in real time: FETCHING=> SUCCESS or ERROR
TC-09	FR-09	Verify graceful Error Handling (User Not Found).	1. Input an invalid username. 2. Click search	Prevents app crash. Displays ERROR: USER_NOT_FOUND in red terminal text.
TC-10	FR-09	Verify error handling during network dropouts.	1. Disconnect network/internet connection. 2. Attempt searching a profile.	Displays a clear fallback error banner state: ERROR: CONNECTION_FAILED.

06 Screenshots & Walkthrough

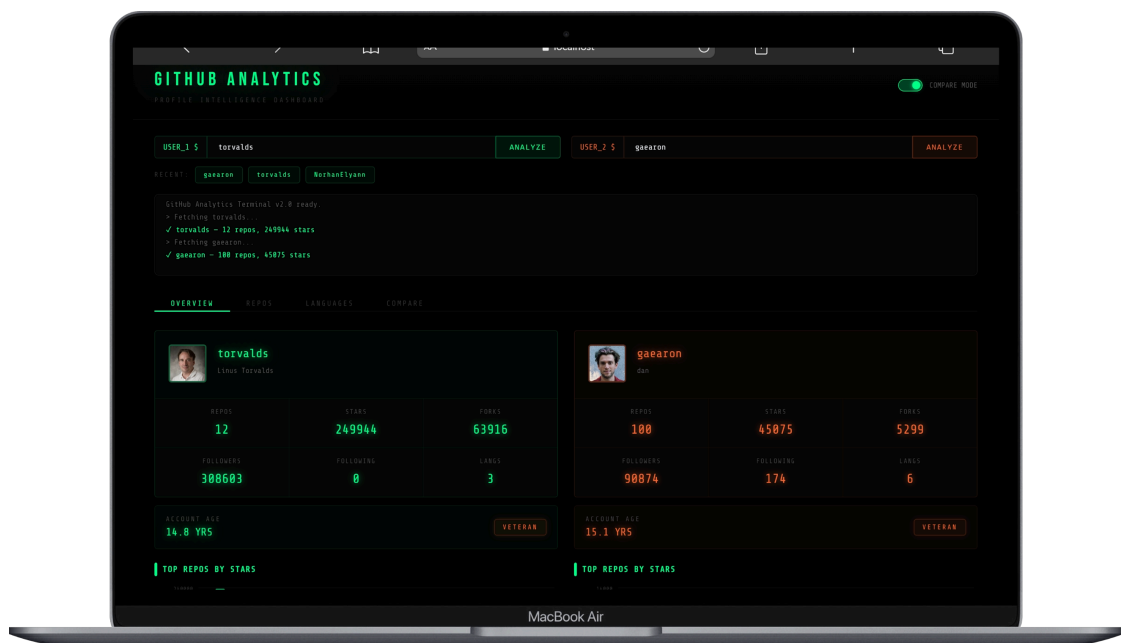
This section walks through each screen of the dashboard with a short explanation of its purpose

6.1 Overview — Single user mode



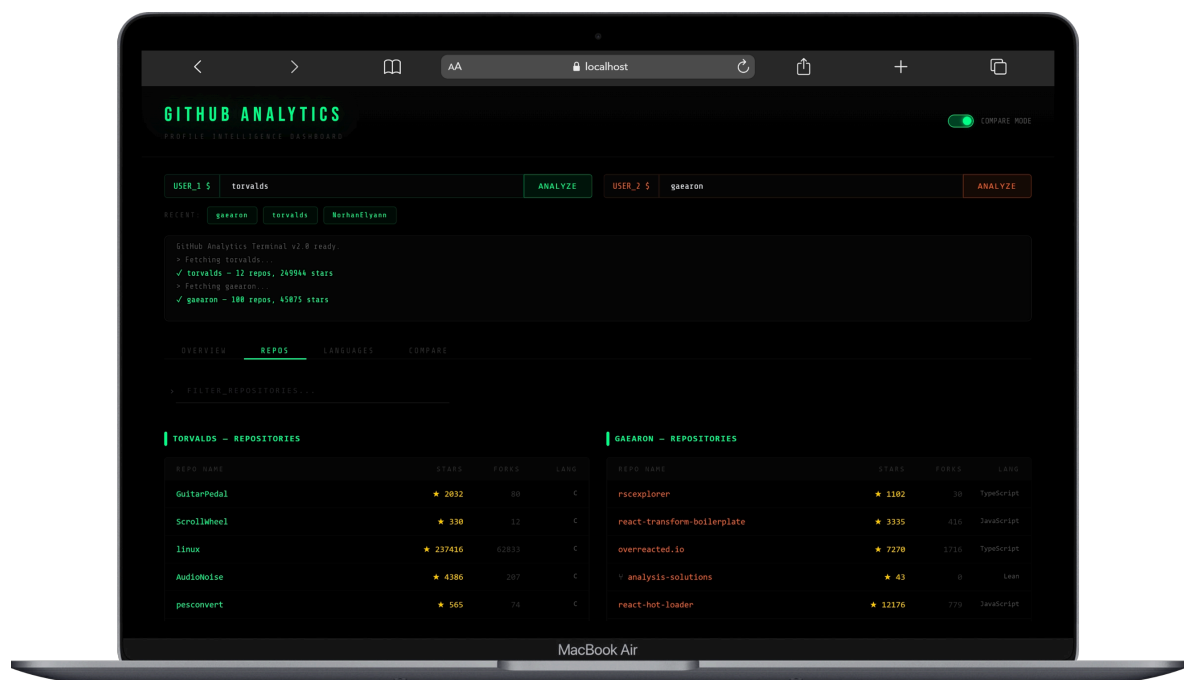
The default landing view. The user types a single GitHub username and clicks ANALYZE. The Terminal log confirms the fetch, and the Overview tab displays the profile card with avatar, name, repo/star/fork counts, followers, following, language count, and account age

6.2 Overview — Compare mode (two profiles)



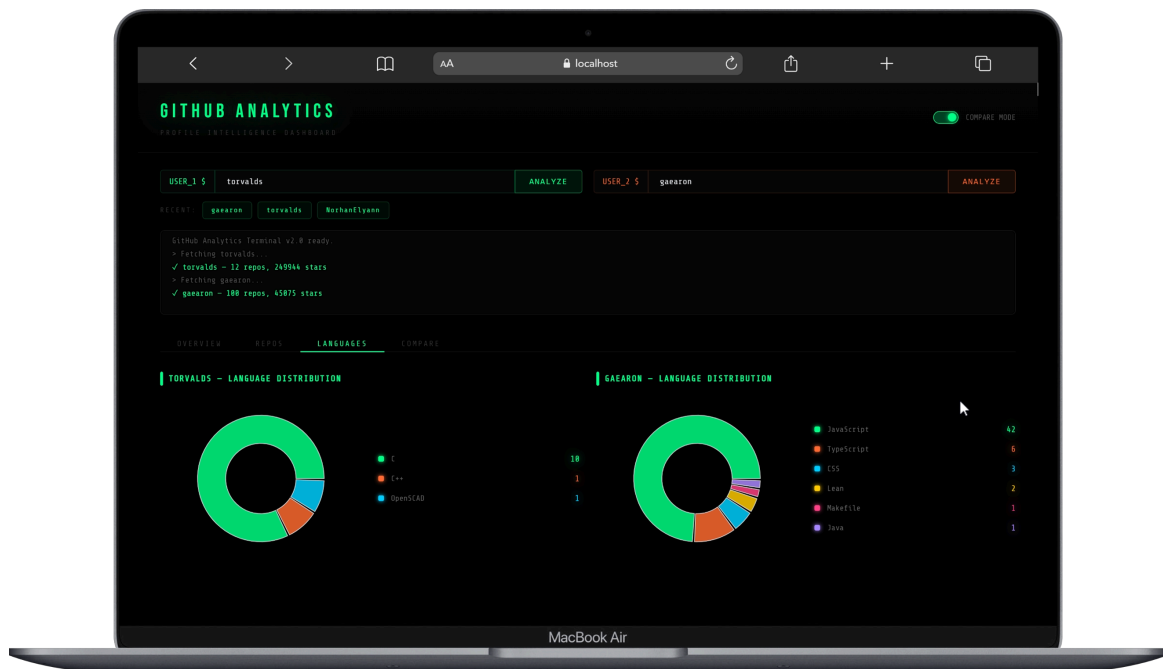
When Compare Mode is enabled, two independent search fields appear (USER_1 / USER_2). Once both profiles are analyzed, their Overview cards render side by side in matching accent colors (green and orange) for instant visual comparison.

6.3 Repos tab



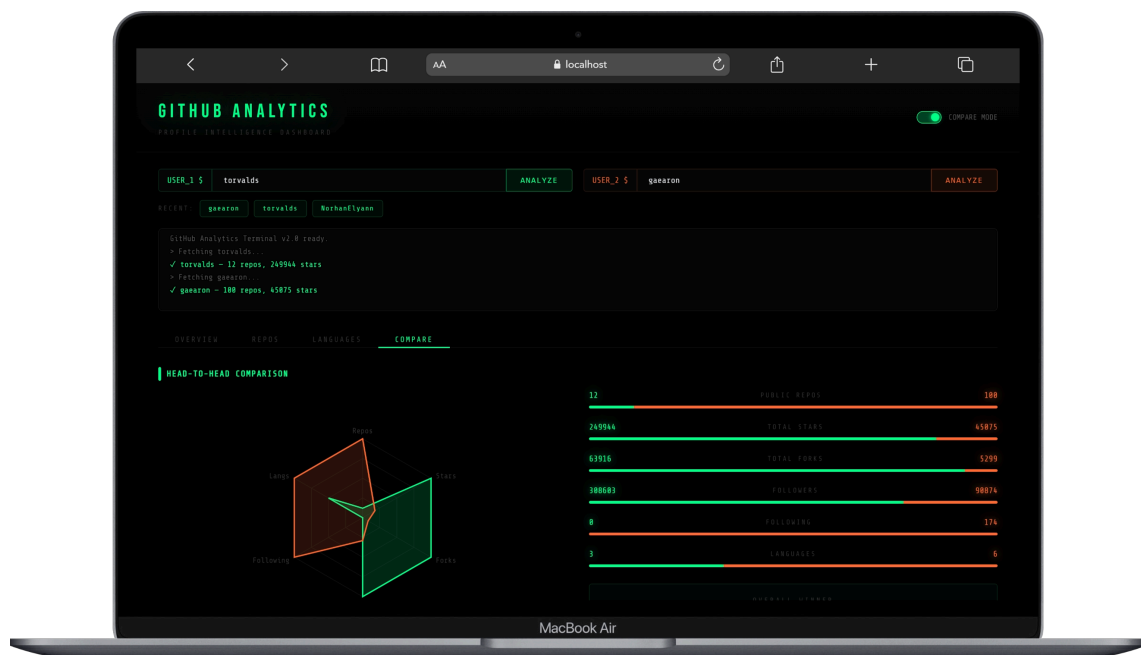
Lists every repository for each user in a sortable table with Stars, Forks, and primary Language. A live filter field lets the user narrow the list by typing a repository name.

6.4 Languages tab



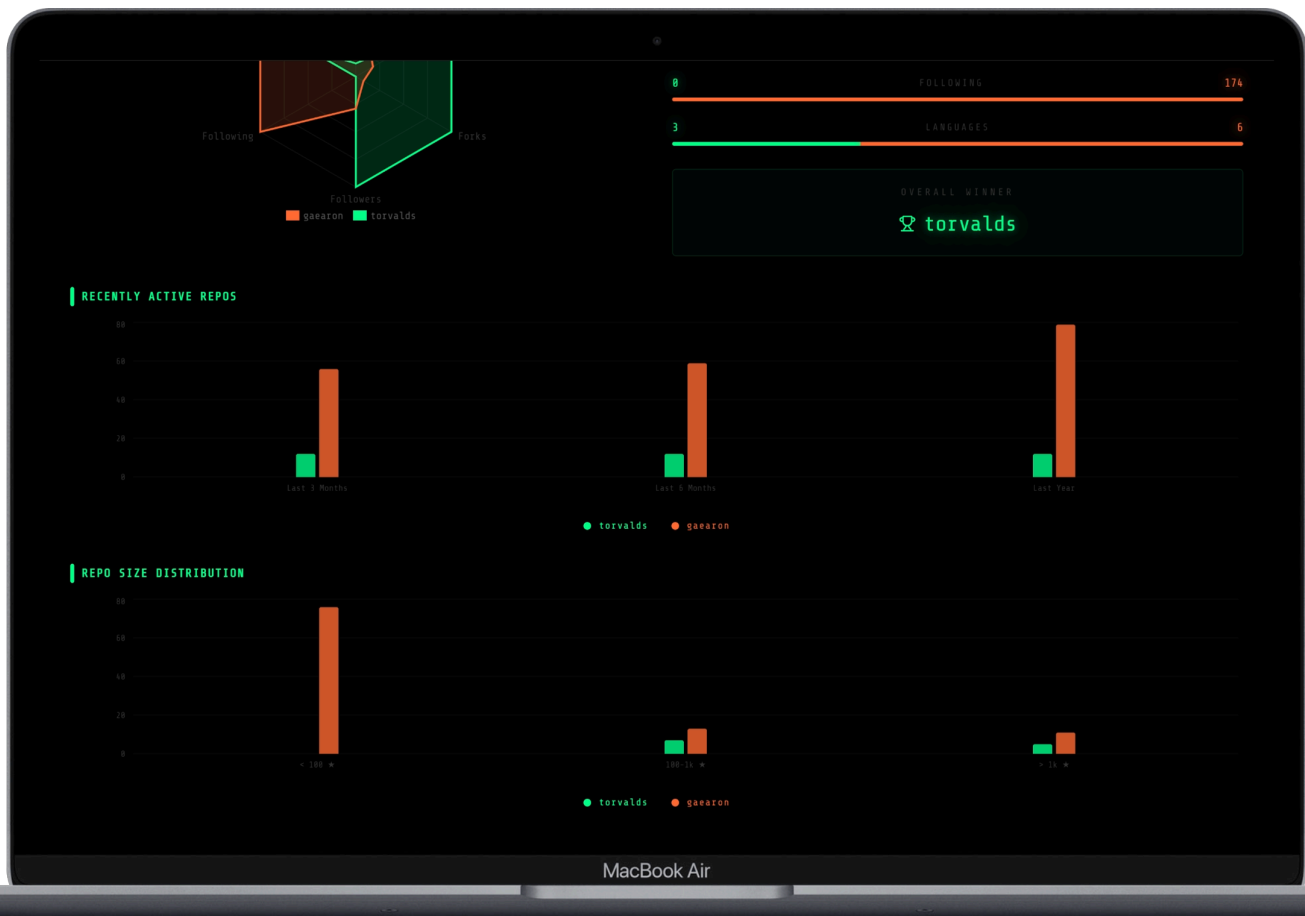
Shows each user's programming language distribution as a donut chart with a color-coded legend listing repo counts per language, making it easy to see each developer's primary tech stack at a glance.

6.5 Compare tab — Head-to-head comparison



The core comparison view. Radar Chart plots both users across six dimensions while Head-to-Head bars down each metric numerically and declare an Overall Winner.

6.6 Compare tab — Recently active repos & repo size distribution



Additional comparison charts shown further down the Compare tab: a bar chart of repository activity over the last 3 months, 6 months, and year, and a breakdown of repository sizes by star-count buckets — giving a deeper view of each user's recent output and project scale.

07 Conclusion

_ GitHub Analytics Dashboard achieved its core goal of providing a fast, visual tool for analyzing and comparing GitHub profiles. The team handled all the technical complexity — GitHub API integration, data processing, and chart rendering — so that the end user only needs to type a username and click ANALYZE, with no coding knowledge or manual API calls required on their part.

_ By project completion, all 12 planned features outlined in section 3.1 were implemented and verified , including profile analysis, repository browsing, language distribution charts, search history and the side-by-side Compare Mode with Winner scoring.

_ The team of 5 developers, working in parallel across separate branches with a clear dependency structure (shared UI components first, feature tabs second), delivered the project without major blockers. The main technical risk identified during planning — GitHub API rate limiting and CORS restrictions — was resolved early by running the project on a local Vite server rather than opening it directly as a static file.

_ The final product meets all functional and non-functional requirements defined in sections 2.4 and 2.5: data loads in under 3 seconds on a good connection, all error states are handled gracefully with clear Terminal log feedback, and the codebase remains organized and maintainable for future iterations.

_ The features listed under section 3.2 (Contribution Graph, Pinned Repos, etc.) remain valid candidates for a future version of the dashboard.